

Stopping Multi-Cluster Drift: ArgoCD vs Flux



Published on February 2, 2025



Webstack
Builders

Table of Contents

A Brief History: Why Not Federation?	3
Multi-Cluster Architectures	4
Common Patterns	4
Consistency vs Flexibility Spectrum	5
Fleet Management Tools	7
Tool Comparison	7
ArgoCD ApplicationSets	9
Flux Multi-Cluster	11
Configuration Templating	14
Kustomize for Multi-Cluster	15
Helm with Fleet Values	18
Drift Detection and Prevention	21
Detecting Configuration Drift	21
Policy-Based Prevention	22
Operational Patterns	25
Progressive Rollout Across Fleet	25
Fleet-Wide Operations	28
Conclusion	30



Running multiple Kubernetes clusters is easy. Keeping them consistent is hard.

As organizations grow, they accumulate clusters: dev, staging, production, regional deployments, edge locations, disaster recovery. Each cluster starts as a copy of the others, templated from the same base configuration. A platform team I worked with managed 15 clusters across dev, staging, and three production regions. They'd done everything right at the start: infrastructure-as-code, consistent tooling, documented architecture. But two years later, hotfixes had been applied to some clusters and not others, "temporary" manual changes became permanent, and someone upgraded the service mesh in US-East but forgot US-West. The clusters that started identical had become 15 unique configurations – and nobody could tell you what was *intentionally* different versus what had *accidentally* drifted.

WARNING

The hardest part of multi-cluster isn't deploying to many clusters – it's knowing what's supposed to be the same vs. different, and detecting when drift occurs. Without this clarity, you're managing independent configurations, not a fleet.

The multi-cluster management problem isn't primarily about tools. It's about the operating model: how you balance consistency with environment-specific flexibility, how you detect when drift occurs, and how you maintain clarity about what's *supposed* to be the same versus what's *supposed* to be different. This article covers the patterns and tools that make fleet management tractable: configuration templating strategies, GitOps (Git Operations)-based deployment, drift detection, and progressive rollout across clusters.

A Brief History: Why Not Federation?

Kubernetes Federation (KubeFed) was the original attempt to solve multi-cluster management. The idea was appealing: a control plane that could deploy resources across multiple clusters from a single API. You'd create a "FederatedDeployment" and it would propagate to member clusters automatically.

In practice, Federation never gained traction. The v1 implementation was deprecated in 2018, and KubeFed v2 saw limited adoption before being archived in 2022. The problems were fundamental: Federation tried to abstract away cluster differences, but real-world multi-cluster deployments *need* those differences. A one-size-fits-all API couldn't handle the nuanced configuration variations between dev and production, between AWS and on-prem, between US-East and EU-West.



The industry moved toward a different model: GitOps (Git Operations)-based fleet management. Instead of a control plane pushing identical configs everywhere, each cluster pulls its configuration from Git, with templating systems handling the variations. This approach – implemented by tools like ArgoCD ApplicationSets, Flux, and Rancher Fleet – provides the consistency benefits Federation promised while preserving the flexibility real deployments require.

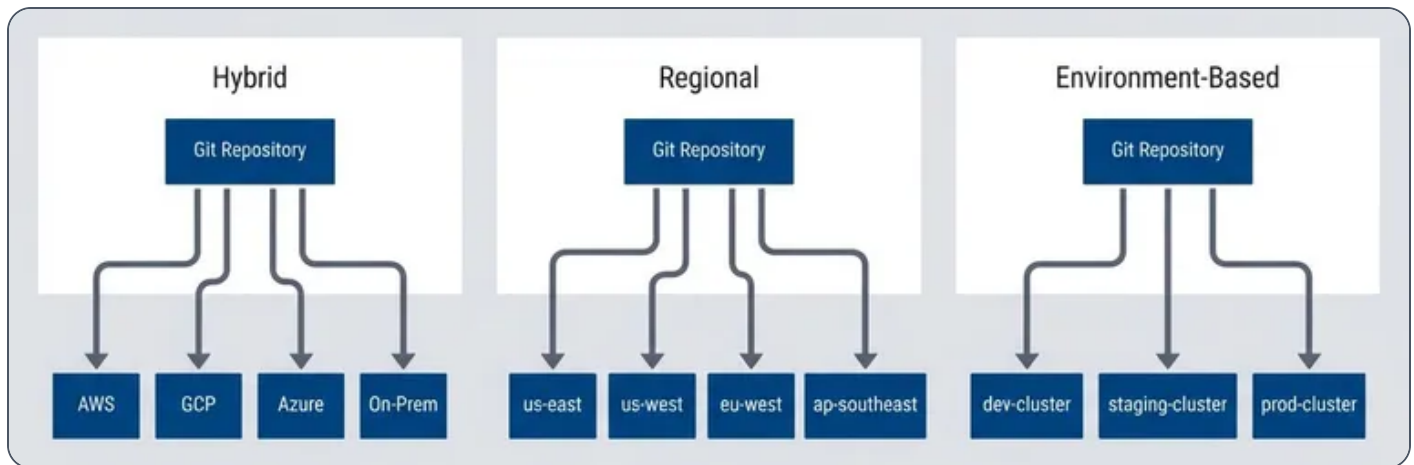
Multi-Cluster Architectures

Common Patterns

Organizations arrive at multi-cluster architectures for different reasons, and the reason shapes the configuration strategy. Understanding *why* you have multiple clusters determines what should be consistent versus different across them.

Environment-based Clusters are the most common pattern: dev, staging, and production running as separate clusters. The configuration model here is straightforward – core add-ons, security policies, and RBAC structure should be identical, while resource quotas, replica counts, and feature flags differ by environment. You want staging to behave like production, just smaller.	
Regional Clusters serve global applications or data sovereignty requirements. US-East, US-West, EU-West, and AP-Southeast might all run the same application versions and security policies, but with region-specific endpoints, compliance settings, and capacity. The challenge here is keeping application deployments synchronized while allowing regional customization.	
Workload isolation Clusters separate concerns by team or workload type: frontend, backend, data platform, ML training. Each cluster shares network policies and security baselines, but node types, add-ons, and resource profiles differ dramatically. An ML training cluster needs GPUs and tolerates batch job failures; a frontend cluster needs fast scaling and zero-downtime deployments.	
Hybrid cloud Clusters span providers: AWS primary, GCP disaster recovery, Azure edge, on-premises for sensitive workloads. Application manifests and security policies stay consistent, but cloud-specific integrations (load balancers, storage classes, IAM bindings) differ by necessity.	





Multi-cluster topology patterns.

Consistency vs Flexibility Spectrum

Every multi-cluster strategy sits somewhere on a spectrum from “all clusters are identical” to “each cluster is independent.” Neither extreme works well in practice.

1

Fully consistent

Means all clusters get exactly the same configuration. This is simple to reason about and guarantees reproducibility, but it’s inflexible. You end up over-provisioning dev and staging to match production, or you can’t tune production for its actual load patterns. This approach only works for small fleets with simple applications.

2

Layered configuration

Uses a base configuration with environment-specific overrides. This is the sweet spot for most organizations. You define what’s shared (security policies, monitoring stack, core add-ons) and what varies (replica counts, resource limits, feature flags). Customize overlays and Helm values files are the typical implementation. The risk is “override explosion”—so many layers of patches that nobody can trace what a cluster actually runs.



3

Policy-governed

Configuration sets guardrails rather than identical configs. Teams can customize their clusters within bounds enforced by OPA/Gatekeeper¹ or Kyverno². This enables innovation and team autonomy, but drift happens within the policy bounds. Debugging becomes harder because clusters are intentionally different.

4

Fully flexible

Means each cluster is independent, with its own GitOps (Git Operations) repo and configuration. Maximum autonomy, maximum chaos. Security risks emerge when teams deviate from baselines. This only works with truly autonomous teams who don't need cross-cluster consistency.

Approach	Consistency	Flexibility	Operational Overhead	Best For
Fully Consistent	Very High	Very Low	Low	Small fleet, simple apps
Layered Config	High	Medium	Medium	Typical enterprise
Policy Governed	Medium	High	High	Platform teams, multi-tenant
Fully Flexible	Low	Very High	Variable	Autonomous teams

Consistency vs flexibility tradeoffs.

INFO

Most organizations land on “layered configuration” with policy guardrails. Pure consistency is too rigid; pure flexibility is chaos. The middle ground – shared base with controlled overrides – balances predictability with practicality.



Fleet Management Tools

The fleet management landscape has matured around a few dominant approaches. All of them embrace GitOps (Git Operations) – storing desired state in Git and reconciling clusters toward that state – but they differ in how they handle multi-cluster targeting, configuration layering, and operational visibility.

Tool Comparison

➤ ArgoCD ApplicationSets

Extends ArgoCD's Application concept with generators that create Applications dynamically based on clusters, Git directories, or custom lists. If you're already running ArgoCD, ApplicationSets are the natural choice. The generator patterns (cluster, list, git, matrix, merge) handle most multi-cluster scenarios elegantly. The limitation is scale: beyond 500 clusters, the ApplicationSet controller can struggle, and you'll need sharding or multiple ArgoCD instances.

➤ Flux

Takes a different approach, building multi-cluster support into its Kustomization hierarchy. Each cluster runs its own Flux controllers pulling from Git, with dependencies between Kustomizations enabling layered configuration. Flux excels at lightweight, distributed deployments where each cluster is fairly autonomous. The trade-off is less centralized visibility – there's no built-in UI showing fleet-wide status.

➤ Rancher Fleet

Provides centralized multi-cluster GitOps with a strong UI for fleet visibility. The "bundle" concept groups related resources for deployment across cluster groups. It's pull-based (clusters pull their configs), but the Rancher UI gives operators a single pane of glass. The downside is ecosystem lock-in – Fleet integrates deeply with Rancher's cluster management.

➤ Google Config Sync (part of Anthos)

Offers deep GKE integration with built-in policy controller and namespace hierarchy management. If your fleet is GKE-centric, Config Sync provides tight integration with Google Cloud's IAM and audit logging. For multi-cloud or on-prem clusters, it's less flexible.

➤ Crossplane

Approaches fleet management differently – as a Kubernetes-native control plane extension for both infrastructure and configuration. You define Compositions that abstract cluster provisioning and configuration together. This is powerful for platform teams building internal developer platforms, but the learning curve is steep.



Tool	Model	Best For	Scale Ceiling	Learning Curve
ArgoCD ApplicationSets	Pull-based, centralized	Teams already using ArgoCD	~500 clusters	Medium
Flux	Pull-based, distributed	CLI-first, GitOps-native teams	1000+ clusters	Medium- High
Rancher Fleet	Pull-based, central UI	Rancher ecosystem users	~500 clusters	Low- Medium
Config Sync	Pull-based, GKE- native	GKE-centric organizations	GKE limits	Low

Fleet management tool comparison.

INFO

Crossplane deserves mention but serves a different purpose – it’s a control plane for provisioning infrastructure **and** configuration together, not purely a fleet deployment tool. If you’re building an internal developer platform where cluster provisioning and app deployment are unified, Crossplane is worth evaluating. For pure configuration management across existing clusters, stick with the GitOps (Git Operations) tools above.

ArgoCD ApplicationSets

ApplicationSets are ArgoCD’s answer to multi-cluster deployment. The core concept is a **generator** that produces a list of parameters, combined with a **template** that stamps out Applications using those parameters. The cluster generator is the most common – it queries ArgoCD’s cluster secrets and creates one Application per matching cluster.

The example below deploys platform services across all production and staging clusters. The cluster generator selects clusters by label (`env: production` or `env: staging`), and the template creates Applications that pull from different Git branches based on environment. Each cluster’s Application syncs from a cluster-specific



path in the repository.

argocd-applicationset.yaml

```

1  apiVersion: argoproj.io/v1alpha1
2  kind: ApplicationSet
3  metadata:
4    name: platform-services
5    namespace: argocd
6  spec:
7    generators:
8      - clusters:
9        selector:
10         matchLabels:
11           env: production
12         values:
13           revision: main
14      - clusters:
15        selector:
16         matchLabels:
17           env: staging
18         values:
19           revision: staging
20  template:
21    metadata:
22      name: "platform-services-{{name}}"
23      labels:
24        env: "{{metadata.labels.env}}"
25        region: "{{metadata.labels.region}}"
26    spec:
27      project: platform
28      source:
29        repoURL: https://github.com/org/platform-config
30        targetRevision: "{{values.revision}}"
31        path: "clusters/{{name}}/platform-services"
32        kustomize:
33          patches:
34            - target:
35              kind: Deployment
36              patch: |
37                - op: replace

```



```
38         path: /spec/replicas
39         value: {{metadata.labels.replicas}}
40     destination:
41       server: "{{server}}"
42       namespace: platform-services
43     syncPolicy:
44       automated:
45         prune: true
46         selfHeal: true
47     syncOptions:
48       - CreateNamespace=true
49       - PrunePropagationPolicy=foreground
```

ApplicationSet using cluster generator with environment-based targeting.

The magic is in the cluster labels. ArgoCD stores cluster connection details as Kubernetes Secrets with special labels. The cluster generator reads these secrets, extracts both connection info and custom labels, then passes them to the template. You can embed cluster-specific values (like replica counts) directly in the cluster secret's labels, making the ApplicationSet template fully generic.

argocd-cluster-secret.yaml

```
1  apiVersion: v1
2  kind: Secret
3  metadata:
4    name: us-east-prod
5    namespace: argocd
6    labels:
7      argocd.argoproj.io/secret-type: cluster
8      env: production
9      region: us-east
10     replicas: "5"
11  type: Opaque
12  stringData:
13    name: us-east-prod
14    server: https://us-east-prod.k8s.example.com
15    # config contains bearerToken and tlsClientConfig (omitted for brevity)
```

Cluster secret with labels for ApplicationSet targeting.



Flux Multi-Cluster

Flux takes a fundamentally different approach: instead of a central controller generating Applications, each cluster runs its own Flux controllers that pull configuration from Git. Multi-cluster management emerges from how you structure your Git repository and how Flux Kustomizations depend on each other.

The key abstraction is the Flux **Kustomization** (not to be confused with Kustomize's `kustomization.yaml` – Flux borrowed the name but it's a different resource). A Flux Kustomization points to a path in a Git repository and reconciles those manifests to the cluster. The `dependsOn` field creates ordering: base configs apply first, then environment overlays, then cluster-specific configs.

flux-kustomization-hierarchy.yaml

```
1  # Base Kustomization applied to all clusters
2  apiVersion: kustomize.toolkit.fluxcd.io/v1
3  kind: Kustomization
4  metadata:
5    name: platform-base
6    namespace: flux-system
7  spec:
8    interval: 10m
9    sourceRef:
10     kind: GitRepository
11     name: platform-config
12     path: ./base/platform
13     prune: true
14 ---
15 # Environment-specific overlay
16 apiVersion: kustomize.toolkit.fluxcd.io/v1
17 kind: Kustomization
18 metadata:
19   name: platform-production
20   namespace: flux-system
21 spec:
22   interval: 10m
23   dependsOn:
24     - name: platform-base
25   sourceRef:
26     kind: GitRepository
27     name: platform-config
```



```

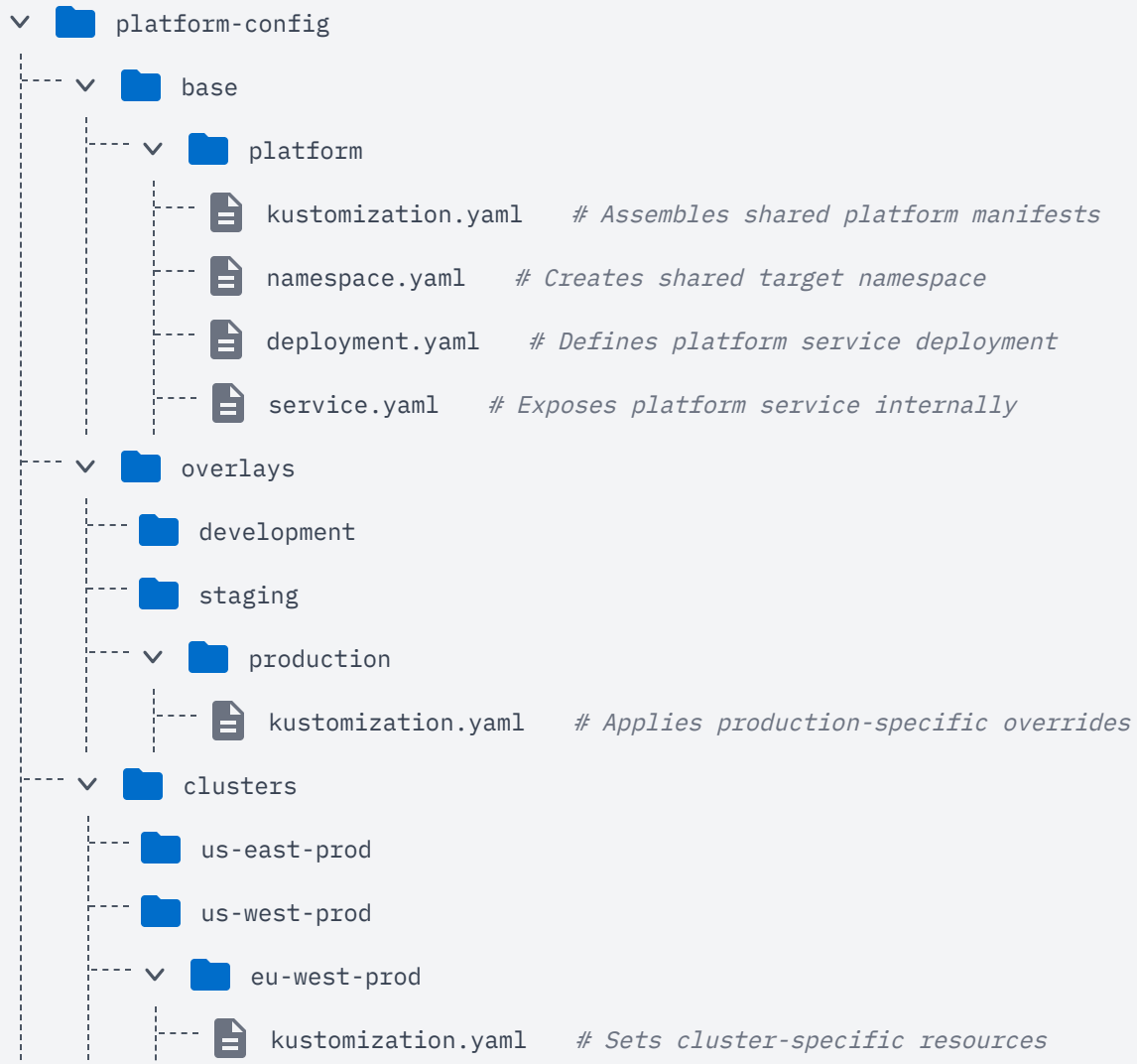
28     path: ./overlays/production
29     prune: true
30     patches:
31     - patch: |
32         - op: replace
33           path: /spec/replicas
34           value: 5
35       target:
36         kind: Deployment
37         labelSelector: "tier=platform"
38 ---
39 # Cluster-specific configuration
40 apiVersion: kustomize.toolkit.fluxcd.io/v1
41 kind: Kustomization
42 metadata:
43   name: cluster-config
44   namespace: flux-system
45 spec:
46   interval: 10m
47   dependsOn:
48   - name: platform-production
49   sourceRef:
50     kind: GitRepository
51     name: platform-config
52     path: ./clusters/${CLUSTER_NAME}
53     prune: true
54     postBuild:
55       substituteFrom:
56       - kind: ConfigMap
57         name: cluster-vars
58       - kind: Secret
59         name: cluster-secrets

```

Flux Kustomization hierarchy with base, overlay, and cluster-specific layers.

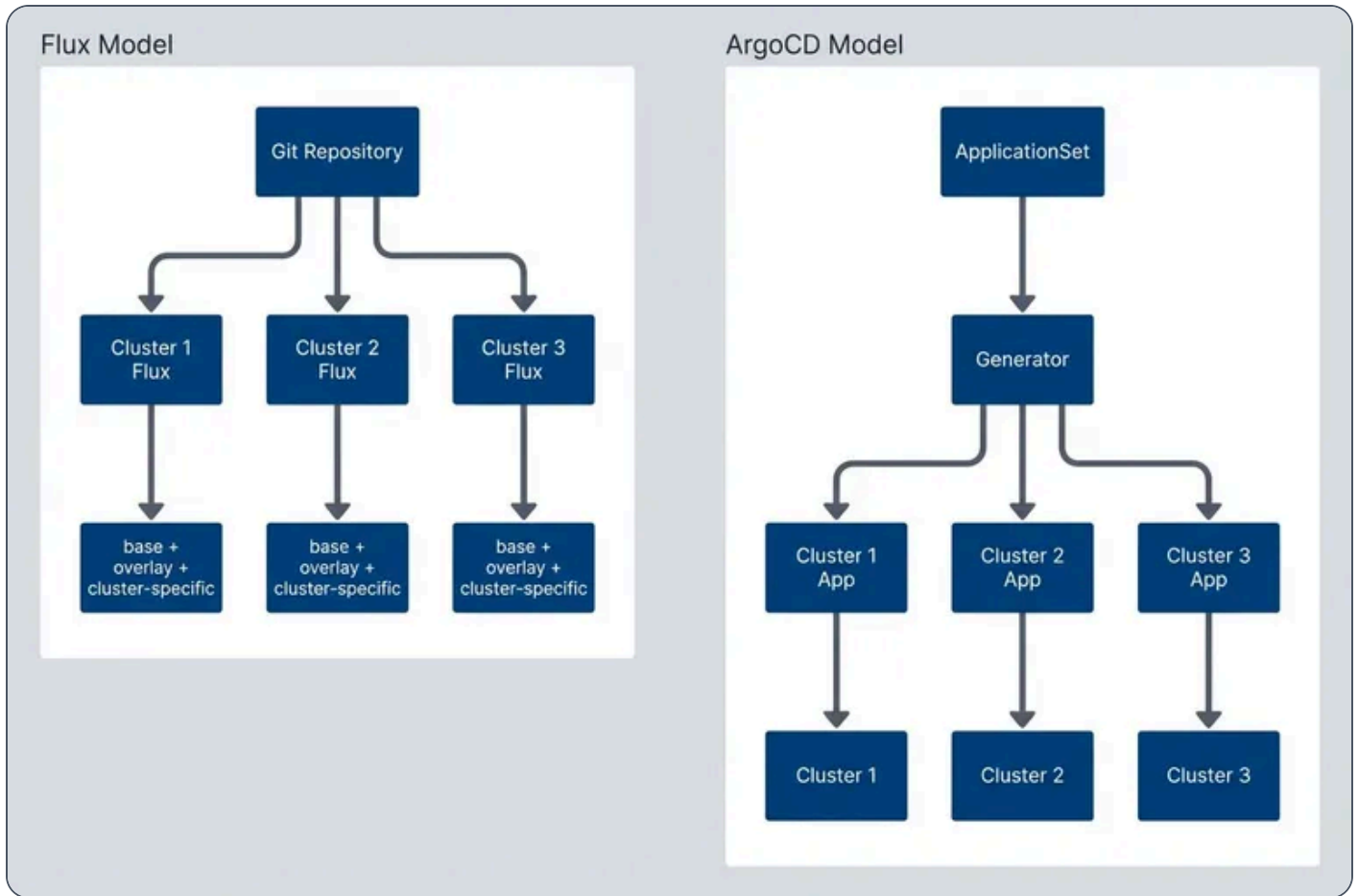
The repository structure mirrors the Kustomization hierarchy. Base configurations define the canonical resources. Environment overlays patch those resources for dev/staging/production. Cluster-specific directories handle the last-mile customization – things like cluster-specific ingress hostnames or region-specific resource limits. The `postBuild.substituteFrom` feature lets you inject cluster-specific values from ConfigMaps or Secrets at reconciliation time, avoiding the need to duplicate paths for every cluster.





Repository structure for Flux multi-cluster configuration.





ArgoCD generates Applications centrally - Flux controllers pull independently.

✓ SUCCESS

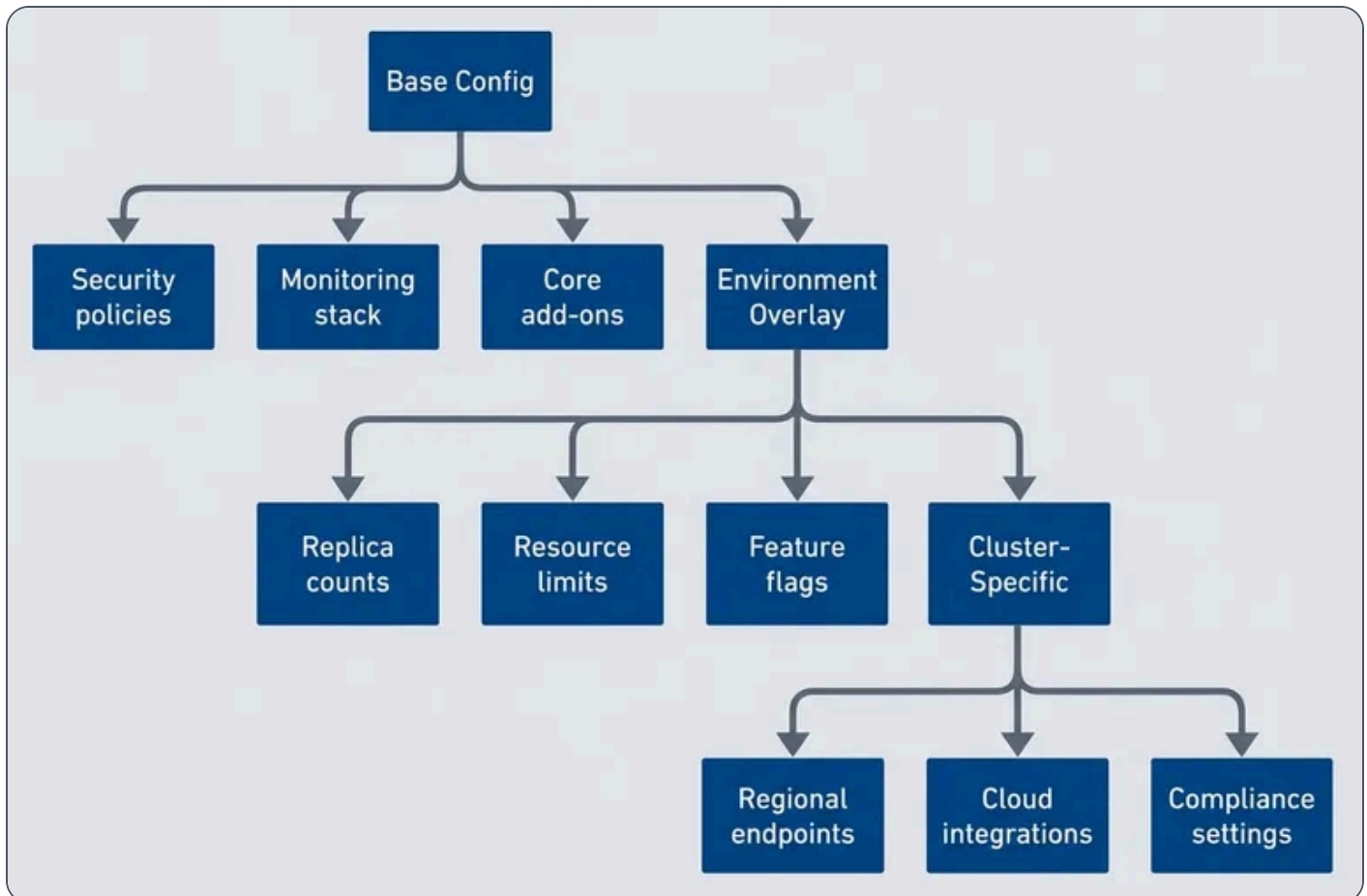
ArgoCD ApplicationSets excel at generating many similar applications with cluster-specific values. Flux's Kustomization hierarchy excels at layered configuration with clear inheritance. Choose based on your team's mental model.

Configuration Templating

Once you've chosen a fleet management tool, the next decision is how to generate cluster-specific configurations from shared definitions. The fleet tool handles *where* configs deploy; the templating strategy handles *what* varies between clusters.



The two dominant approaches are Kustomize (patch-based) and Helm (template-based), though alternatives like Jsonnet³ and CUE⁴ exist for organizations with more complex needs.



Configuration layering from fleet-wide to cluster-specific.

Kustomize for Multi-Cluster

Kustomize uses a layered approach: a base directory contains canonical manifests, and overlay directories patch those manifests for specific environments or clusters. There's no templating – just strategic merge patches and JSON patches that modify the base resources.

The typical structure has three layers: base (shared across all clusters), environment overlays (development, staging, production), and cluster-specific overlays for last-mile customization. Each layer's

`kustomization.yaml` references the previous layer and adds patches.



kustomize-base.yaml

```

1  # Base configuration (base/kustomization.yaml)
2  apiVersion: kustomize.config.k8s.io/v1beta1
3  kind: Kustomization
4  resources:
5    - namespace.yaml
6    - deployment.yaml
7    - service.yaml
8    - configmap.yaml
9  commonLabels:
10   app.kubernetes.io/managed-by: fleet

```

Base kustomization with shared resources.

The production overlay references the base and layers on deployment-specific settings – replica counts, pinned image tags, resource requests and limits, and merged ConfigMap values. Kustomize’s `patches` field uses JSON Patch operations to surgically modify individual fields without rewriting the entire manifest.

kustomize-production-overlay.yaml

```

1  # Production overlay (overlays/production/kustomization.yaml)
2  apiVersion: kustomize.config.k8s.io/v1beta1
3  kind: Kustomization
4  resources:
5    - ../../base
6  namePrefix: prod-
7  replicas:
8    - name: api-server
9      count: 5
10 images:
11   - name: api-server
12     newTag: v2.1.0
13 patches:
14   - target:
15       kind: Deployment
16       name: api-server
17     patch: |
18       - op: add
19         path: /spec/template/spec/containers/0/resources

```



```

20         value:
21           requests:
22             cpu: "500m"
23             memory: "512Mi"
24           limits:
25             cpu: "2000m"
26             memory: "2Gi"
27   configMapGenerator:
28   - name: app-config
29     behavior: merge
30     literals:
31     - LOG_LEVEL=info
32     - ENABLE_DEBUG=false

```

Production overlay with resource limits and config.

The final layer targets a specific cluster. It references the production overlay and adds region-aware patches – injecting the correct database endpoint and constraining pod scheduling to the matching availability zone via `nodeSelector` .

kustomize-cluster-overlay.yaml

```

1  # Cluster-specific (clusters/us-east-prod/kustomization.yaml)
2  apiVersion: kustomize.config.k8s.io/v1beta1
3  kind: Kustomization
4  resources:
5  - ../../overlays/production
6  patches:
7  - target:
8      kind: ConfigMap
9      name: app-config
10    patch: |
11      - op: add
12        path: /data/REGION
13        value: us-east-1
14      - op: add
15        path: /data/DB_HOST
16        value: us-east-prod.db.example.com
17  - target:
18      kind: Deployment

```



```
19     name: api-server
20     patch: |
21       - op: replace
22         path: /spec/template/spec/nodeSelector
23         value:
24           topology.kubernetes.io/region: us-east-1
```

Cluster-specific overlay with regional configuration.

The advantage of Kustomize is that there's no templating language to learn – it's native Kubernetes YAML with patches. The disadvantage is that complex transformations require many patches, and the final rendered output isn't obvious from reading the source files.

Helm with Fleet Values

Helm takes the opposite approach: templates with variable substitution. A single chart can deploy to any cluster by providing different values files. For fleet management, you layer values files just like Kustomize overlays: base values, environment values, cluster-specific values.

helm-base-values.yaml

```
1  # Base values (values/base.yaml)
2  replicaCount: 2
3  image:
4    repository: myapp/api
5    tag: latest
6    pullPolicy: IfNotPresent
7  resources:
8    requests:
9      cpu: 100m
10     memory: 128Mi
11  config:
12    logLevel: debug
13    enableMetrics: true
```

Base Helm values for all environments.



The production values file overrides the base defaults with deployment-grade settings – higher replica counts, a pinned image tag, explicit resource limits, and a PodDisruptionBudget to maintain availability during rolling updates.

helm-production-values.yaml

```
1  # Production values (values/production.yaml)
2  replicaCount: 5
3  image:
4    tag: v2.1.0
5    pullPolicy: Always
6  resources:
7    requests:
8      cpu: 500m
9      memory: 512Mi
10   limits:
11     cpu: 2000m
12     memory: 2Gi
13  config:
14    logLevel: info
15    enableMetrics: true
16  podDisruptionBudget:
17    enabled: true
18    minAvailable: 3
```

Production values with higher resources and PDB.

The cluster-specific file adds the final layer of customization – scaling replicas for a high-traffic region, wiring in the regional database endpoint, and using `nodeSelector` and `tolerations` to pin workloads to dedicated node pools.

helm-cluster-values.yaml

```
1  # Cluster-specific values (values/clusters/us-east-prod.yaml)
2  replicaCount: 8 # Higher traffic region
3  config:
4    region: us-east-1
5    dbHost: us-east-prod.db.example.com
6  nodeSelector:
```



```

7     topology.kubernetes.io/region: us-east-1
8   tolerations:
9     - key: "dedicated"
10       operator: "Equal"
11       value: "api"
12       effect: "NoSchedule"

```

Cluster-specific values for high-traffic region.

Helmfile orchestrates this layering, merging values files in order and injecting environment variables for cluster-specific paths.

helmfile.yaml

```

1   environments:
2     development:
3       values:
4         - values/development.yaml
5     staging:
6       values:
7         - values/staging.yaml
8     production:
9       values:
10        - values/production.yaml
11
12  releases:
13    - name: api-server
14      namespace: production
15      chart: ./charts/api-server
16      values:
17        - values/base.yaml
18        - values/{{ .Environment.Name }}.yaml
19        - values/clusters/{{ requiredEnv "CLUSTER_NAME" }}.yaml

```

Helmfile for multi-cluster deployment orchestration.

The key thing to get right with layered values is merge order. Helm performs a shallow merge by default, so nested maps in a later values file replace – rather than merge with – the same key from an earlier file. If your base defines `resources.requests.cpu` and a cluster file only sets `resources.requests.memory`, you



lose the CPU request entirely. Use Helmfile's `mergeStrategy: strategic` or restructure values to keep overrides at leaf keys to avoid silent misconfigurations that only surface under load.

Approach	Pros	Cons	Best For
Kustomize overlays	Native K8s, no templating	Patch complexity grows	Simple environment overrides
Helm values hierarchy	Powerful templating	Values file explosion	Complex apps with many variants
Jsonnet	Programmable, DRY	Steep learning curve	Large-scale platforms
CUE	Type-safe, validation	Newer, less ecosystem adoption	Configuration validation focus

Configuration templating approaches.

INFO

The “values file explosion” problem is real: base + environment + region + cluster = many files to manage. Combat this with clear naming conventions, validation in CI, and documentation of which values are set where.

Drift Detection and Prevention

Drift – the divergence between desired state in Git and actual state in clusters – is the silent killer of fleet consistency. It happens gradually: a hotfix applied directly to production, a debugging session that left a ConfigMap modified, an operator who “just needed to bump the replica count.” Each change seems harmless, but they accumulate until your fleet is a collection of undocumented variations that nobody fully understands.



Detecting Configuration Drift

GitOps (Git Operations) tools provide built-in drift detection. ArgoCD compares the rendered manifests from Git against the live cluster state on every sync interval (default 3 minutes). When they differ, the Application shows as “OutOfSync” in the UI and emits metrics. Flux does the same comparison through its reconciliation loop, surfacing drift through the `Ready` condition on Kustomization resources.

The key configuration choice is whether to **detect** drift or **auto-remediate** it. ArgoCD’s `selfHeal: true` setting automatically reverts any manual changes, forcing the cluster back to the Git-defined state. This is powerful for preventing drift but can be dangerous if someone made a legitimate emergency change.

argocd-sync-policy.yaml

```
1  apiVersion: argoproj.io/v1alpha1
2  kind: Application
3  metadata:
4    name: platform-services
5  spec:
6    syncPolicy:
7      automated:
8        prune: true      # Delete resources removed from Git
9        selfHeal: true   # Revert manual changes automatically
10   syncOptions:
11     - Validate=true    # Validate manifests before applying
```

ArgoCD sync policy with auto-remediation.

For fleet-wide visibility, export drift metrics to your monitoring system. ArgoCD exposes `argocd_app_info` metrics with sync status labels. Flux exposes reconciliation status through `gotk_reconcile_condition`. Build dashboards that show drift across all clusters at a glance, and alert when clusters stay out-of-sync for more than a few reconciliation cycles.

PromQL

```
1  # ArgoCD: count of out-of-sync applications by cluster
2  count by (dest_server) (argocd_app_info{sync_status="OutOfSync"})
3
4  # Flux: count of failed reconciliations by cluster
```



```
5 count by (exported_namespace) (gotk_reconcile_condition{type="Ready", status="False"})
```

Prometheus queries for fleet-wide drift visibility.

Policy-Based Prevention

Detection tells you when drift happens; prevention stops it from happening in the first place. Kubernetes admission controllers can intercept API requests before resources are created or modified, rejecting changes that violate fleet policies.

The two main policy engines are OPA/Gatekeeper and Kyverno. For drift prevention, the common patterns are:

1

Require fleet management labels

Reject resources that lack labels indicating they're fleet-managed. This catches resources created outside the GitOps flow.

2

Require GitOps annotations

Reject modifications to resources that should only be changed through Git. ArgoCD adds tracking annotations; policies can require their presence.

3

Restrict manual changes in production

Allow direct kubectl access in dev/staging but block it in production namespaces.

Here's a Gatekeeper constraint that requires fleet management labels on all workloads:

gatekeeper-required-labels.yaml

```
1  apiVersion: templates.gatekeeper.sh/v1
2  kind: ConstraintTemplate
3  metadata:
4    name: k8srequiredlabels
5  spec:
6    crd:
```




```

7      spec:
8        names:
9          kind: K8sRequiredLabels
10       validation:
11         openAPIV3Schema:
12           type: object
13           properties:
14             labels:
15               type: array
16               items:
17                 type: string
18       targets:
19         - target: admission.k8s.gatekeeper.sh
20           rego: |
21             package k8srequiredlabels
22             violation[{"msg": msg}] {
23               provided := {label | input.review.object.metadata.labels[label]}
24               required := {label | label := input.parameters.labels[_]}
25               missing := required - provided
26               count(missing) > 0
27               msg := sprintf("Missing required labels: %v", [missing])
28             }
29       ---
30     apiVersion: constraints.gatekeeper.sh/v1beta1
31     kind: K8sRequiredLabels
32     metadata:
33       name: fleet-managed-resources
34     spec:
35       match:
36         kinds:
37           - apiGroups: ["apps"]
38             kinds: ["Deployment", "StatefulSet", "DaemonSet"]
39     parameters:
40       labels:
41         - "app.kubernetes.io/managed-by"
42         - "fleet.example.com/cluster"

```

Gatekeeper constraint requiring fleet management labels.

And a Kyverno policy that rejects changes to resources without GitOps (Git Operations) annotations:



kyverno-gitops-only.yaml

```
1  apiVersion: kyverno.io/v1
2  kind: ClusterPolicy
3  metadata:
4    name: require-gitops-annotation
5  spec:
6    validationFailureAction: Enforce
7    background: true
8    rules:
9      - name: check-gitops-source
10        match:
11          any:
12            - resources:
13                kinds:
14                  - Deployment
15                  - Service
16                  - ConfigMap
17                namespaces:
18                  - production
19        validate:
20          message: "Resources in production must be managed by GitOps."
21          pattern:
22            metadata:
23              annotations:
24                argocd.argoproj.io/tracking-id: "*"
```

Kyverno policy enforcing GitOps (Git Operations)-only changes in production.

⚠ WARNING

Drift prevention through admission control is powerful but can block emergency changes. Always have a break-glass procedure – a way for authorized users to make manual changes in emergencies, with automatic detection and follow-up.



Operational Patterns

Day-to-day fleet management involves two distinct challenges: rolling out changes across clusters safely, and performing fleet-wide operations when something goes wrong. Both require thinking about ordering, verification, and rollback.

Progressive Rollout Across Fleet

Never deploy to all production clusters simultaneously. A bad configuration change, a broken image tag, a misconfigured resource limit – any of these can take down your entire fleet if deployed everywhere at once. Progressive rollout deploys to clusters in waves, verifying health between each wave.

The typical wave structure is:

1

Canary cluster

A single non-critical cluster (or a dedicated canary) that receives changes first. If something breaks, the blast radius is minimal.

2

Non-critical production

Secondary regions, lower-traffic clusters, or less critical workloads. These validate the change works at production scale without risking your primary clusters.

3

Primary production

The clusters that matter most. By the time changes reach here, they've been validated in real production conditions.

ArgoCD's ApplicationSet supports progressive rollout through its RollingSync strategy. You assign wave labels to clusters and define steps that sync clusters matching each wave:

```
progressive-rollout-applicationset.yaml
```

```
1  apiVersion: argoproj.io/v1alpha1
2  kind: ApplicationSet
```



```

3  metadata:
4    name: platform-progressive-rollout
5  spec:
6    generators:
7      - list:
8        elements:
9          - cluster: canary-cluster
10            wave: "1"
11          - cluster: prod-secondary
12            wave: "2"
13          - cluster: prod-primary
14            wave: "3"
15  strategy:
16    type: RollingSync
17    rollingSync:
18      steps:
19        - matchExpressions:
20          - key: wave
21            operator: In
22            values: ["1"]
23        - matchExpressions:
24          - key: wave
25            operator: In
26            values: ["2"]
27          maxUpdate: 1
28        - matchExpressions:
29          - key: wave
30            operator: In
31            values: ["3"]
32  template:
33    metadata:
34      name: "platform-{{cluster}}"
35      labels:
36        wave: "{{wave}}"
37    spec:
38      source:
39        repoURL: https://github.com/org/platform-config
40        path: "clusters/{{cluster}}"
41      destination:
42        server: "{{server}}"
43        namespace: platform
44      syncPolicy:

```



```
45     automated:
46     selfHeal: true
```

ApplicationSet with progressive rollout waves.

Between waves, you need automated verification. Argo Rollouts' AnalysisTemplates can query Prometheus for error rates and latency, automatically blocking progression if metrics exceed thresholds:

rollout-analysis.yaml

```
1  apiVersion: argoproj.io/v1alpha1
2  kind: AnalysisTemplate
3  metadata:
4    name: fleet-rollout-check
5  spec:
6    metrics:
7      - name: error-rate
8        interval: 1m
9        successCondition: result[0] < 0.01
10       provider:
11         prometheus:
12           address: http://prometheus:9090
13           query: |
14             sum(rate(http_requests_total{status=~"5.*",cluster="{{args.cluster}}"}[5m]))
15             /
16             sum(rate(http_requests_total{cluster="{{args.cluster}}"}[5m]))
17       - name: latency-p99
18         interval: 1m
19         successCondition: result[0] < 500
20         provider:
21           prometheus:
22             address: http://prometheus:9090
23             query: |
24               histogram_quantile(0.99,
25                 sum(rate(http_request_duration_seconds_bucket{cluster="{{args.cluster}}"}[5m])) by (le)
26                 ) * 1000
```

AnalysisTemplate for automated rollout verification.



Fleet-Wide Operations

Sometimes you need to perform operations across the entire fleet that don't fit the GitOps (Git Operations) model: emergency patching, fleet-wide debugging, state comparison, or coordinated restarts. For these, you need operational tooling that understands cluster ordering and can handle failures gracefully.

The key principles are:

1

Order by criticality

Operate on canary/dev clusters first, production last. If something fails, you find out before touching critical infrastructure.

2

Verify between clusters

Don't blindly proceed. Check health, validate the operation succeeded, wait for stabilization.

3

Have a rollback plan

Know how to undo the operation. For some operations (like config changes), rollback is straightforward. For others (like data migrations), it's not.

4

Respect rate limits

Don't hammer your clusters. Add delays between operations, especially in production.

Operation Type	Strategy	Verification	Rollback
Security patch	Parallel (all at once)	Basic health	Per-cluster
Feature rollout	Progressive waves	Canary metrics	Full fleet
Config change	Sequential	Config validation	Previous config
Add-on upgrade	Progressive	Integration tests	Previous version



Fleet operation strategies by type.

✓ SUCCESS

Progressive rollout is essential for production fleet operations. Start with a canary cluster, verify health, then proceed to non-critical clusters, and finally critical clusters. Each wave should have clear success criteria before proceeding.

Conclusion

The shift from Kubernetes Federation to GitOps (Git Operations)-based fleet management reflects a fundamental insight: multi-cluster consistency isn't about making clusters identical, it's about making differences *intentional* and *documented*. Federation failed because it tried to abstract away cluster variations. Modern fleet tools succeed because they embrace variations while providing guardrails.

The practical path forward has four components. First, document your fleet topology explicitly – which clusters exist, what each serves, what should be shared versus different. This documentation becomes the foundation for your templating strategy, whether you choose Kustomize overlays or Helm values hierarchies.

Second, choose tools that match how your team thinks. ArgoCD ApplicationSets work well if you think in terms of “deploy this app to these clusters.” Flux's Kustomization hierarchy fits teams who think in terms of “base config plus environment overrides plus cluster-specific tweaks.” Fighting your team's mental model leads to workarounds that undermine the system.

Third, implement drift detection from day one, before clusters have had time to diverge. The choice between auto-remediation and alert-and-review depends on your risk tolerance – auto-heal everything in dev, be more cautious in production. Either way, you need visibility into what's drifting and why.

Finally, treat fleet-wide changes as first-class operations with their own patterns: progressive rollout through canary clusters, automated verification between waves, and clear rollback procedures. The overhead of wave-based deployment pays for itself the first time it catches a bad change before it reaches your primary production clusters.



 INFO

Fleet management maturity is measured by how confidently you can answer: “What’s different between these clusters, and is that difference intentional?” If you can’t answer quickly, start with documentation. Tools amplify good processes; they can’t substitute for them.

One area this article hasn’t covered in depth is secrets management across clusters – synchronizing credentials, certificates, and API keys while maintaining proper access controls. Tools like External Secrets Operator, Sealed Secrets, and HashiCorp Vault each solve this differently. That’s a topic for its own deep dive.

- 1 OPA (Open Policy Agent) is an open-source, general-purpose policy engine that unifies policy enforcement across the stack, using a declarative language called Rego to define complex rules. Gatekeeper is a specialized project that integrates OPA into Kubernetes – it acts as a validating admission controller, intercepting requests to the Kubernetes API and checking them against OPA policies before resources are created or modified. ↩
- 2 Kyverno is a Kubernetes-native policy engine. Unlike Gatekeeper, it doesn’t require learning Rego; policies are written in standard YAML. It can validate, mutate (modify), and generate resources, as well as verify container image signatures. ↩
- 3 Jsonnet is a data templating language from Google, designed to generate JSON (and by extension, YAML). It’s a full programming language with functions, conditionals, and imports, making it powerful for complex configuration generation. The trade-off is that Jsonnet requires learning a new language, and debugging can be challenging when templates become deeply nested. ↩
- 4 CUE (Configure, Unify, Execute) is an open-source data constraint language designed for defining schemas and validating data. It’s a superset of JSON – any valid JSON is also valid CUE – but adds type constraints, default values, and logical unification. CUE isn’t part of Kubernetes or the CNCF, but it’s widely used in the Kubernetes ecosystem for configuration validation and generation. ↩



Copyright © 2025 Webstack Builders, Inc.

The text, diagrams, and images in this work are licensed under CC BY-NC 4.0

All code samples in this article are licensed under the MIT License. Feel free to use, modify, and distribute them in any project.

<https://www.webstackbuilders.com/articles/kubernetes-multi-cluster-fleet-management-configuration>

